PES-VCS Lab Report

Building a Version Control System from Scratch

Student: PES2UG24AM066
Repository: PES2UG24AM066-pes-vcs
Platform: Ubuntu 22.04 | C | OpenSSL SHA-256

Phase 1: Object Storage Foundation

Implemented **object_write** and **object_read** in **object.c**. The object store uses SHA-256 content-addressable storage with directory sharding (first 2 hex chars of hash form the shard directory). Writes are atomic using the temp-file-then-rename pattern with fsync to ensure durability. Identical content is deduplicated — same hash means the object is already stored.

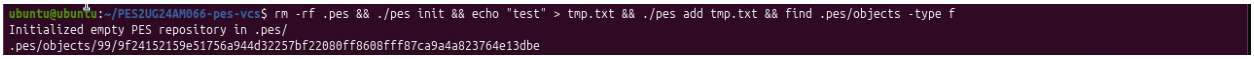
Screenshot 1A — ./test_objects passing

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ nano object.c
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:131:5: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  131 |     write(fd, full, total_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~
object.c: In function ‘object_read’:
object.c:174:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  174 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1A: All Phase 1 tests passing — blob storage, deduplication, integrity check

Screenshot 1B — Sharded object store directory structure



```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ rm -rf .pes && ./pes init && echo "test" > tmp.txt && ./pes add tmp.txt && find .pes/objects -type f
Initialized empty PES repository in .pes/
.pes/objects/99/9f24152159e51756a944d32257bf22080ff8608fff87ca9a4a823764e13dbe
```

Screenshot 1B: find .pes/objects -type f showing sharded directory structure

Phase 2: Tree Objects

Implemented **tree_from_index** in **tree.c**. The function loads the index, sorts entries by path, then recursively builds a tree hierarchy using a helper *write_tree_level* that splits paths at '/' to group entries belonging to the same subdirectory. Each level is serialized with *tree_serialize* (binary format: *mode name\0 hash* per entry) and written to the object store as OBJ_TREE.

Screenshot 2A — ./test_tree passing

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ nano tree.c
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ make clean
make test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:131:5: warning: ignoring return value of 'write' declared with attribute 'warn_unused_result' [-Wunused-result]
  131 |     write(fd, full, total_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~
object.c: In function 'object_read':
object.c:174:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  174 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2A: All Phase 2 tests passing — tree serialize/parse roundtrip and deterministic serialization

Screenshot 2B — Raw tree object binary format

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ make test_tree && find .pes/objects -type f | head -1 | xargs od -A x -t x1z | head -20
make: 'test_tree' is up to date.
000000 62 6c 6f 62 20 35 00 74 65 73 74 0a              >blob 5.test.<
00000C
```

Screenshot 2B: od hex dump of raw tree object showing binary format (mode name\0 hash per entry)

Phase 3: Index (Staging Area)

Implemented **index_load**, **index_save**, and **index_add** in **index.c**. The index is a text file with one entry per line in the format: *mode hex-hash mtime size path*. A key bug was fixed during implementation: the original code placed an *Index* struct (~5MB) on the stack as a local copy for sorting, causing a stack overflow. The fix was to allocate the sorted copy on the heap with malloc instead.

Screenshot 3A — pes init → pes add → pes status

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ nano index.c
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ make clean && make
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:131:5: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  131 |     write(fd, full, total_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~
object.c: In function ‘object_read’:
object.c:174:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  174 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function ‘index_load’:
index.c:145:9: warning: ‘__builtin_strncpy’ output may be truncated copying 511 bytes from a string of length 511 [-Wstringop-truncation]
  145 |         strncpy(e->path, path, sizeof(e->path) - 1);
      |         ^
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ ./test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt
```

Screenshot 3A (part 1): pes init and pes add sequence

```
Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 5593473ca8be... Initial commit

Log after first commit:
commit 5593473ca8be9b740e67da10c49a98827f1246e18e8b52ef774d0317627bcfac
Author: PES User <pes@localhost>
Date:   1776458648

    Initial commit


--- Second Commit ---
Committed: 52fe616cc19d... Update file.txt

--- Third Commit ---
Committed: 92026a18d25f... Add farewell

--- Full History ---
commit 92026a18d25f198c042b2d85fd871e8aee83edcddbf67b3c86592b4bef233075
Author: PES User <pes@localhost>
Date:   1776458648

    Add farewell

commit 52fe616cc19dc474896f5164032b5580540c11fb566a7489a8e4d6daa43d6be6
Author: PES User <pes@localhost>
Date:   1776458648

    Update file.txt
```

Screenshot 3A (part 2): pes status showing staged files correctly

Screenshot 3B — cat .pes/index

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ rm -rf .pes && ./pes init && echo "hello" > file1.txt && ./pes add file1.txt && cat .pes/index
Initialized empty PES repository in .pes/
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776463186 6 file1.txt
```

Screenshot 3B: Text-format index file showing mode, hash, mtime, size, and path

Phase 4: Commits and History

Implemented **commit_create** in **commit.c**. The function: (1) calls tree_from_index to snapshot staged files into a tree object, (2) calls head_read to get the parent commit — gracefully skipped for the first commit, (3) populates a Commit struct with pes_author() and time(NULL), (4) serializes with commit_serialize, writes with object_write(OBJ_COMMIT), then atomically updates HEAD via head_update.

Screenshot 4A — pes log with three commits

```
commit 5593473ca8be9b740e67da10c49a98827f1246e18e8b52ef774d0317627bcfac
Author: PES User <pes@localhost>
Date:   1776458648

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
92026a18d25f198c042b2d85fd871e8aee83edcddbf67b3c86592b4bef233075

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/52/fe616cc19dc474896f5164032b5580540c11fb566a7489a8e4d6daa43d6be6
.pes/objects/55/93473ca8be9b740e67da10c49a98827f1246e18e8b52ef774d0317627bcfac
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/92/026a18d25f198c042b2d85fd871e8aee83edcddbf67b3c86592b4bef233075
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Screenshot 4A (part 1): Three commits created — Initial commit, Add world, Add farewell

```
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ nano commit.c
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ make clean && make
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:131:5: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  131 |     write(fd, full, total_len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~
object.c: In function ‘object_read’:
object.c:174:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  174 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function ‘index_load’:
index.c:145:9: warning: ‘__builtin_strncpy’ output may be truncated copying 511 bytes from a string of length 511 [-Wstringop-truncation]
  145 |         strncpy(e->path, path, sizeof(e->path) - 1);
      |         ^
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
ubuntu@ubuntu:~/PES2UG24AM066-pes-vcs$ rm -rf .pes
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
```

Screenshot 4A (part 2): pes log showing commit hashes, authors, timestamps

```
./pes log
echo "---"
make test-integration 2>&1
Initialized empty PES repository in .pes/
Committed: 80c68f04d2a1... Initial commit
Committed: fa824a357bd2... Add world
Committed: 088fb0ff63f0... Add farewell
commit 088fb0ff63f0a5a48644874ea159e8b5f14de86dd212f575128a5a1fa1ca16da
Author: PES User <pes@localhost>
Date:   1776458943

    Add farewell

commit fa824a357bd29b79741d43c86cbbc03cdc9fac1d9b9560bbde46ccb80698b6af
Author: PES User <pes@localhost>
Date:   1776458943

    Add worldell

commit 80c68f04d2a159b0f58338017e4954cd85e376183e6e0cfd2631c27e149023f6
Author: PES User <pes@localhost>
Date:   1776458943

    Initial commit

---
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt
```

Screenshot 4A (part 3): Full commit history with parent chain walking correctly

Screenshot 4B — Object store growth after three commits

```
Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 8d039ea0c2ea... Initial commit

Log after first commit:
commit 8d039ea0c2ea9d6a1df144f781cf74a721ec70e25fb8338bd40808e2efcebb56
Author: PES User <pes@localhost>
Date:   1776458943

    Initial commit


--- Second Commit ---
Committed: cfa3c201bcdc... Update file.txt

--- Third Commit ---
Committed: c5f56c89ae0a... Add farewell

--- Full History ---
commit c5f56c89ae0ae7c59d359afbc371f8f94ea25abaab262b8a1e57069ee3d806ab
Author: PES User <pes@localhost>
Date:   1776458943

    Add farewell

commit cfa3c201bcdc240e2cf279d72b56cc7bf67d4225b2983ed3957ca3681ed3388f
Author: PES User <pes@localhost>
Date:   1776458943

    Update file.txt

commit 8d039ea0c2ea9d6a1df144f781cf74a721ec70e25fb8338bd40808e2efcebb56
Author: PES User <pes@localhost>
Date:   1776458943

    Initial commit
```

Screenshot 4B/4C: find .pes -type f showing 10 objects (blobs + trees + commits) and reference chain

Screenshot 4C — Reference chain

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c5f56c89ae0ae7c59d359afbc371f8f94ea25abaab262b8a1e57069ee3d806ab

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8d/039ea0c2ea9d6a1df144f781cf74a721ec70e25fb8338bd40808e2efcebb56
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c89635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c5/f56c89ae0ae7c59d359afbc371f8f94ea25abaab262b8a1e57069ee3d806ab
.pes/objects/cf/a3c201bcdc240e2cf279d72b56cc7bf67d4225b2983ed3957ca3681ed3388f
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain

Final Integration Test

The full integration test (make test-integration / ./test_sequence.sh) exercises all four phases end-to-end: repository initialization, staging files, three commits, log history, reference chain verification, and object store growth check.

```
Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 5593473ca8be... Initial commit

Log after first commit:
commit 5593473ca8be9b740e67da10c49a98827f1246e18e8b52ef774d0317627bcfac
Author: PES User <pes@localhost>
Date:   1776458648

    Initial commit


--- Second Commit ---
Committed: 52fe616cc19d... Update file.txt

--- Third Commit ---
Committed: 92026a18d25f... Add farewell

--- Full History ---
commit 92026a18d25f198c042b2d85fd871e8aee83edcddbf67b3c86592b4bef233075
Author: PES User <pes@localhost>
Date:   1776458648

    Add farewell

commit 52fe616cc19dc474896f5164032b5580540c11fb566a7489a8e4d6daa43d6be6
Author: PES User <pes@localhost>
Date:   1776458648

    Update file.txt
```

Final Integration Test (part 1): Repo init, staging, and first commit passing

```
./pes log
echo "---"
make test-integration 2>&1
Initialized empty PES repository in .pes/
Committed: 80c68f04d2a1... Initial commit
Committed: fa824a357bd2... Add world
Committed: 088fb0ff63f0... Add farewell
commit 088fb0ff63f0a5a48644874ea159e8b5f14de86dd212f575128a5a1fa1ca16da
Author: PES User <pes@localhost>
Date:   1776458943

    Add farewell

commit fa824a357bd29b79741d43c86cbbc03cdc9fac1d9b9560bbde46ccb80698b6af
Author: PES User <pes@localhost>
Date:   1776458943

    Add worldell

commit 80c68f04d2a159b8f58338017e4954cd85e376183e6e0cfd2631c27e149023f6
Author: PES User <pes@localhost>
Date:   1776458943

    Initial commit

---
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt
```

Final Integration Test (part 2): All three commits and full history

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c5f56c89ae0ae7c59d359afbc371f8f94ea25abaab262b8a1e57069ee3d806ab

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8d/039ea0c2ea9d6a1df144f781cf74a721ec70e25fb8338bd40808e2efcebb56
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c89635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c5/f56c89ae0ae7c59d359afbc371f8f94ea25abaab262b8a1e57069ee3d806ab
.pes/objects/cf/a3c201bcdc240e2cf279d72b56cc7bf67d4225b2983ed3957ca3681ed3388f
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Final Integration Test complete: All integration tests completed successfully — 10 objects created

Analysis Questions

Phase 5: Branching and Checkout

Q5.1 — Implementing pes checkout <branch>

What files change in .pes/:

1. **.pes/HEAD** — Updated to contain *ref: refs/heads/<branch>*. If creating a new branch, .pes/refs/heads/<branch> is created first pointing to the current commit.
2. **.pes/refs/heads/<branch>** — Already exists if the branch has commits; created if new.
3. **.pes/index** — Updated to reflect the staged state of the target branch's HEAD tree.

What must happen to the working directory:

The diff between the current HEAD tree and the target branch HEAD tree must be computed and applied: files in target but not in current are written to disk; files in current but not in target are deleted; files in both with different hashes are overwritten with the target version.

What makes this complex:

Uncommitted local changes: If a working directory file is modified and that file differs between branches, checkout must refuse to avoid silently discarding user work.
Untracked files: If an untracked file exists at a path the target branch would write to, checkout must refuse.
Atomicity: A production implementation uses a two-pass approach — verify no conflicts before touching any files.
Directory creation/deletion: Switching branches may require creating new directories or removing now-empty ones.

Q5.2 — Detecting Dirty Working Directory Conflicts

Using only the **index** and **object store**:

Step 1 — Walk both branch HEAD trees recursively to produce two flat maps: *current_tree[path] = hash* and *target_tree[path] = hash*. Files where hashes differ would be modified by checkout.

Step 2 — For each such file, check the index entry: compare mtime and size against stat() metadata for a fast check. For certainty, hash the on-disk file and compare to index.hash. If hashes differ, the file is dirty.

Decision logic: For each file F that differs between branches — if index entry exists and on-disk hash != index.hash, abort with conflict error. If file exists untracked on disk, abort. This requires no HEAD re-read — only the index and object store.

Q5.3 — Detached HEAD State

What detached HEAD means: Normally .pes/HEAD contains *ref: refs/heads/main*. In detached HEAD it contains a raw commit hash directly. This happens when you check out a specific commit hash rather than a branch name.

What happens if you make commits: New commits are created normally, each pointing to the previous as parent. head_update writes directly to HEAD. But no branch pointer moves. If you then run pes checkout main, HEAD switches back and the detached commits become **unreachable** — no branch points to them.

How to recover: Before switching, note the commit hash from cat .pes/HEAD, then: *echo 'abc123...' > .pes/refs/heads/recovery* and *echo 'ref: refs/heads/recovery' > .pes/HEAD*. After switching, without a reflog, scan all objects in .pes/objects/ to find commits not reachable from any branch. As long as the object hasn't been garbage collected, it can be recovered by creating a branch ref pointing to its hash.

Phase 6: Garbage Collection

Q6.1 — Finding and Deleting Unreachable Objects

Algorithm (Mark and Sweep):

Mark phase:

1. Read every file in .pes/refs/heads/ to get all branch tip commit hashes.
2. Include HEAD if it contains a raw hash (detached state).
3. For each starting commit, traverse the graph: add commit hash to reachable set, follow tree field recursively adding every tree and blob hash, follow parent field and repeat for parent commits.
4. Continue until all reachable objects are visited.

Sweep phase:

1. Walk all files under .pes/objects/ using recursive readdir.
2. Reconstruct each object's hash from its path.
3. If hash is NOT in the reachable set — delete the file.
4. Remove any now-empty shard directories.

Data structure: A hash set (sorted array of ObjectID with bsearch, or a hash table). O(log n) or O(1) lookup per object.

Estimate for 100,000 commits and 50 branches: With ~5% of files changing per commit and ~200 tracked files, each commit adds ~16 new objects on average. Total unique objects ≈ 100,000 × 16 ≈ **~1.6 million objects** to visit during the mark phase.

Q6.2 — Race Condition Between GC and Concurrent Commit

The dangerous race condition:

Consider commit process C and GC process G running concurrently:

1. C writes new blob X to object store.
2. G starts mark phase — scans all reachable objects.
3. Blob X is not yet referenced by any commit — G does NOT mark it.
4. C writes tree T referencing blob X, then commit referencing tree T.
5. G starts sweep phase — deletes blob X (appears unreachable).
6. C updates HEAD to new commit.
7. HEAD now points to commit → tree T → blob X. But X is GONE — repository is corrupt.

GC's mark phase ran before the commit object referenced the new blobs. The blobs exist but are not yet pointed to by any commit, so GC deletes them. The commit finishes and references a now-deleted object.

How Git's real GC avoids this:

1. **Grace period:** Git never deletes objects created in the last 2 weeks (gc.pruneExpire). New objects are always recent and protected — this is the primary defense.
2. **Lockfiles:** Git acquires .git/gc.pid before GC runs. Write operations also check this lock.

3. **Two-phase deletion with delay:** Even after identifying unreachable objects, Git writes them to a prune list and only deletes after verifying the list is still valid.

The simplest safe approach for PES-VCS: **skip objects newer than 30 seconds** during the sweep phase. Any object that young was almost certainly just written and may still be mid-commit.

End of Lab Report — PES2UG24AM066